Práctica 2: Procesamiento de datos almacenados en HBase

Curso: Ciencia de Datos Escalable · **Entorno:** Amazon EMR con HBase · **Autor:** Raymond Negron
Repositorio: https://github.com/raymond1242/MAE-CDE-hbase

> Las tablas de resultados de este informe se generan a partir de los archivos de `results/` con `python3 src/build_report.py`. Esos archivos se obtuvieron en el clúster EMR siguiendo `docs/02_step_by_step.md`.

---

1. Entorno y datos

Elemento	Valor
Clúster EMR	`j-00430072CCSGFQ8WWNLK` · `emr-7.14.0` en `us-east-2` (Ohio), Amazon Linux 2023 · 1 nodo primario + 1 nodo core
HBase	`HBase 2.6.4-amzn-2` : 1 master activo, 1 RegionServer, servidor Thrift en el puerto 9090
Tabla	`emp` , familias `personal` y `professional`
Columnas usadas	`personal:name` , `personal:city` , `professional:designation` , `professional:salary`
Dataset	300 empleados sintéticos (`data/employees.csv` , semilla 42)

Diferencias con la estructura esperada: ninguna. La tabla usa exactamente los nombres de familias y calificadores del enunciado. Si se usara otra tabla, el único cambio sería `COLUMN_MAP` en `src/config.py` .

Por qué un dataset sintético: permite controlar el esquema exacto que pide la práctica, no contiene datos personales reales y es reproducible (misma semilla, mismo archivo). Incluye a propósito 28 problemas de calidad (salarios faltantes, no numéricos, negativos o con espacios; ciudades con mayúsculas o espacios inconsistentes; celdas ausentes) para demostrar su tratamiento explícito. El detalle está en `data/README.md` .

2. Cómo se obtienen los datos de HBase

Mecanismo principal: Python + `happybase` a través del servidor Thrift de HBase. EMR instala el componente `hbase-thrift-server` en el nodo primario (puerto 9090). Thrift es un protocolo RPC que expone la API de HBase a lenguajes distintos de Java. La librería `happybase` abre la conexión y ejecuta un `scan` sobre la tabla `emp` solicitando solo las cuatro columnas necesarias. Esa proyección se aplica en el servidor y las filas llegan en lotes de 500. Cada fila se convierte de bytes a texto UTF-8, y las celdas inexistentes quedan como valores faltantes (`None`). Código: `src/hbase_readers.py` (`read_via_thrift`).

Método alternativo (si Thrift no estuviera disponible): exportar la salida del HBase Shell. Se ejecuta `scan 'emp'` en modo no interactivo (`hbase shell -n`), se guarda la salida en `results/raw/scan_emp.txt` y Python la interpreta línea por línea (`read_via_shell_export`). Este método solo necesita el cliente `hbase` , que siempre está presente en el clúster, y deja la exportación como evidencia auditable. Se probaron ambos métodos sobre la misma tabla y producen resultados idénticos.

Cada extracción guarda una copia fiel de lo leído (`results/emp_snapshot.csv`) y sus metadatos:

item	value
method	thrift (happybase)
server / file	`localhost:9090`
table	`emp`
rows_read	300
extracted_at (UTC)	2026-10-24T06:32:20+00:00
python	3.9.25

3. Actividad A: verificación de datos

1. `describe 'emp'` muestra las dos familias de columnas (`personal` , `professional`) con `VERSIONS => 1` . Los calificadores no aparecen porque HBase no tiene un esquema fijo por debajo de la familia.
2. `scan 'emp'` muestra los registros. La salida completa está en `results/raw/scan_emp.txt` .
3. Para verificar las columnas necesarias se contó cuántas filas tienen cada celda. La diferencia con el total corresponde a valores faltantes:

command	rows
`count 'emp'`	300
`count 'emp', {COLUMNS => ['personal:name']}`	300
`count 'emp', {COLUMNS => ['personal:city']}`	297
`count 'emp', {COLUMNS => ['professional:designation']}`	298
`count 'emp', {COLUMNS => ['professional:salary']}`	292

4. Actividad B: resultados

Todos los indicadores se calculan sobre los datos leídos de HBase, después de aplicar las reglas de limpieza de la sección 4.6.

4.1 ¿Cuántos empleados hay registrados en cada ciudad?

4.2 ¿Qué porcentaje del total corresponde a cada ciudad?

city	employees	percentage
Lima	67	22.33
Bogota	56	18.67
Santiago	56	18.67
Quito	45	15.00
Buenos Aires	44	14.67
Madrid	29	9.67
UNKNOWN	3	1.00

Respuesta: Lima concentra más empleados (67, 22.33% del total). `UNKNOWN` agrupa los registros sin ciudad.

4.3 ¿Cuál es el salario promedio, mínimo y máximo?

metric	value
total_records	300
missing_salaries	8
invalid_salaries	8
fixed_salaries	2
valid_salaries	284
avg_salary	48647.18
min_salary	18000.00
max_salary	129000.00
median_salary	45600.00
stdev_salary	18578.62

Respuesta: salario promedio 48647.18, mínimo 18000.00 y máximo 129000.00 USD, calculados sobre 284 salarios válidos de 300 registros (8 faltantes y 8 inválidos excluidos; 2 corregidos al quitar espacios).

4.4 ¿Cuántos empleados hay por cargo?

designation	employees	percentage
Data Analyst	72	24.00
Software Engineer	63	21.00
Administrative Assistant	48	16.00
Data Scientist	39	13.00
Project Manager	33	11.00
Accountant	28	9.33
Director	15	5.00
UNKNOWN	2	0.67

Respuesta: el cargo más frecuente es Data Analyst (72 empleados, 24.00%).

4.5 Si hay suficientes registros, ¿qué ciudad presenta el mayor salario promedio?

Se exige un mínimo de 5 salarios válidos por ciudad para que su promedio entre en la comparación (`MIN_SALARIES_PER_CITY` en `src/config.py`). `UNKNOWN` (sin ciudad) no participa en el ranking.

rank	city	employees	valid_salaries	avg_salary	min_salary	max_salary	meets_minimum
1	Madrid	29	26	57676.92	29300.00	129000.00	true
2	Santiago	56	54	51181.48	21800.00	106700.00	true
3	Lima	67	63	50506.35	19900.00	100600.00	true
4	Buenos Aires	44	42	46657.14	19500.00	81000.00	true
5	Bogota	56	53	46628.30	18400.00	94200.00	true
6	Quito	45	43	41472.09	18000.00	85600.00	true
—	UNKNOWN	3	3	52100.00	41200.00	59500.00	false

Respuesta: sí hay registros suficientes (mínimo 5 salarios válidos por ciudad). La ciudad con mayor salario promedio es Madrid (57676.92 USD, 26 salarios válidos).

4.6 Tratamiento de valores faltantes e inválidos

Reglas aplicadas en `src/indicators.py`:

- **Salario:** se quitan los espacios al inicio y al final. Se acepta solo un número decimal simple y positivo (`45000`, `45000.50`). Se excluyen de las estadísticas salariales: la celda inexistente, el texto no numérico (`N/A`, `pending`), los valores `0` o negativos y los formatos ambiguos como `4,250` (puede ser 4250 o 4.25 según la configuración regional). El empleado sigue contando en los conteos por ciudad y por cargo.
- **Ciudad y cargo:** las variantes de mayúsculas y espacios (`LIMA`, `' lima '`) se agrupan con la forma más frecuente (`Lima`). Las celdas inexistentes se agrupan como `UNKNOWN`, que se mantiene en los conteos para que los porcentajes sumen 100%.

field	problem	treatment	rows
city	inconsistent case/whitespace	normalized to 'Bogota'	1
city	inconsistent case/whitespace	normalized to 'Lima'	3
city	inconsistent case/whitespace	normalized to 'Santiago'	1
city	missing value	grouped as UNKNOWN	3
designation	missing value	grouped as UNKNOWN	2
salary	cell professional:salary does not exist	excluded from salary statistics	8
salary	not a number	excluded from salary statistics	5
salary	not positive	excluded from salary statistics	3
salary	surrounding whitespace removed	numeric value used	2

Detalle fila por fila: `results/05_data_quality.csv`.

4.7 Lo que HBase responde por sí solo frente a lo que calcula Python

Las consultas de `hbase/04_activity_b_queries.hbase` cuentan empleados con filtros del lado del servidor (`SingleColumnValueFilter`). HBase compara bytes exactos, así que no reconoce `LIMA` ni `' lima '` como `Lima`. Tampoco interpreta números: el filtro `salary > 90000` compara texto.

column	value	hbase_exact_filter	python_normalized	difference
personal:city	Lima	64	67	3
personal:city	Bogota	55	56	1
personal:city	Santiago	55	56	1
personal:city	Quito	45	45	0
personal:city	Buenos Aires	44	44	0
personal:city	Madrid	29	29	0
professional:designation	Data Analyst	72	72	0
professional:designation	Software Engineer	63	63	0
professional:designation	Administrative Assistant	48	48	0
professional:designation	Data Scientist	39	39	0
professional:designation	Accountant	28	28	0
professional:designation	Project Manager	33	33	0
professional:designation	Director	15	15	0

Otras consultas de HBase y filas devueltas:

command	rows
`scan 'emp', {COLUMNS => ['personal:city'], FILTER => "ValueFilter(=, 'substring:lima')"}`	67
`count 'emp'`	300
`scan 'emp', {COLUMNS => ['professional:salary'], LIMIT => 10}`	10
`scan 'emp', {COLUMNS => ['professional:salary'], FILTER => "SingleColumnValueFilter('professional', 'salary', >, 'binary:90000', true, true)"}`	9
`get 'emp', 'E0042'`	1
`scan 'emp', {STARTROW => 'E0100', STOPROW => 'E0105'}`	5

5. Interpretación de resultados

- La plantilla se concentra en Lima (22.3%), Bogotá y Santiago (18.7% cada una), casi el 60% del total. Madrid es la sede más pequeña (9.7%).
- Predominan los perfiles técnicos: Data Analyst y Software Engineer suman el 45%, y los directores solo el 5%.
- El salario promedio (48 647 USD) supera a la mediana (45 600 USD), en un rango de 18 000 a 129 000 USD. Unos pocos cargos de alto salario, como los directores (unos 92 000 USD), sesgan la distribución hacia arriba.
- Madrid tiene el mayor promedio (57 677 USD con 26 salarios válidos, 18.6% sobre el general), aunque es la ciudad con menor proporción de cargos mejor pagados (17%). Esto apunta a un nivel salarial local más alto. Quito tiene el promedio más bajo (41 472 USD).
- La limpieza fue determinante. Se excluyó el 5.3% de los salarios, y sin normalizar las ciudades Lima habría quedado subcontada (64 empleados en lugar de 67). Al ser datos sintéticos, los resultados ilustran el método más que un mercado laboral real.

6. Análisis conceptual

6.1 ¿Qué parte del trabajo realiza HBase y qué parte realiza Python?

HBase almacena y recupera los datos. Guarda las filas distribuidas en el clúster, ordenadas por clave y agrupadas en familias de columnas. Además responde a `put`, `get`, `scan` y `count`, incluidos los filtros y la selección de columnas. Sin embargo, trata cada valor como bytes: no sabe que `salary` es un número ni que `LIMA` y `Lima` son la misma ciudad, y no calcula promedios ni agrupaciones.

Python hace el análisis. Lee las filas desde HBase (happybase vía Thrift), convierte los salarios a números, trata los valores faltantes e inválidos, normaliza las categorías y calcula conteos, porcentajes, promedio, mínimo y máximo. La sección 4.7 lo muestra con dos ejemplos:

- El filtro exacto de HBase cuenta 64 empleados en Lima, mientras que Python cuenta 67.
- El filtro `salary > 90000` de HBase devuelve textos como `N/A`, porque compara texto y no números.

6.2 ¿Por qué `get` es adecuado para recuperar una fila conocida y `scan` para recorrer varias filas?

`get` lee una sola fila a partir de su clave exacta. HBase va directo a esa fila sin recorrer la tabla, así que es la operación adecuada cuando se conoce la clave, por ejemplo `get 'emp', 'E0042'`.

`scan` recorre varias filas: toda la tabla o un rango de claves (`STARTROW` / `STOPROW`), aprovechando que las filas se guardan ordenadas por clave. Devuelve los resultados en lotes y permite filtrar y elegir columnas. Por eso es la operación adecuada para procesar muchas filas o cuando no se conocen las claves. Hacer un `get` por cada fila sería mucho más lento, con una petición por fila, y exigiría conocer todas las claves de antemano.

6.3 ¿Qué limitaciones tendría recuperar toda una tabla grande para procesarla en un único programa?

- **Memoria:** todas las filas tendrían que caber en la RAM de una sola máquina.
- **Tiempo:** los datos llegan de forma secuencial a través de un único cliente, así que el tiempo crece con el tamaño de la tabla.
- **Sin paralelismo:** los datos están repartidos en el clúster, pero un solo programa no aprovecha esa distribución.
- **Fallos:** si el proceso se interrumpe a mitad de una lectura larga, hay que empezar de nuevo.
- **Carga sobre HBase:** un recorrido completo compite con otras aplicaciones que usan la misma tabla.

6.4 ¿Cómo cambiaría la solución con millones de registros procesados de forma distribuida? Papel de Apache Spark

Con millones de registros, el análisis debe repartirse entre varias máquinas en lugar de concentrarse en un solo programa. **Apache Spark**, que puede instalarse en el mismo clúster EMR, cumpliría ese papel:

- **Lectura en paralelo:** lee la tabla de HBase dividida en particiones, una por región, y varios nodos las procesan a la vez.
- **Procesamiento distribuido:** la conversión de salarios, la normalización de ciudades y los agregados (`groupBy("city")` con conteo, promedio, mínimo y máximo) se ejecutan en paralelo. Al programa principal solo llega el resultado final, que es pequeño.
- **Tolerancia a fallos:** si un nodo falla, Spark repite solo la parte perdida, no todo el proceso.

HBase seguiría almacenando y sirviendo los datos. Spark reemplazaría a Python como motor de procesamiento.

7. Reproducibilidad

Todo el flujo está versionado en este repositorio. El paso a paso completo está en `docs/02_step_by_step.md`. Resumen de comandos en el nodo primario de EMR:

```
bash infra/setup_primary.sh && source .venv/bin/activate
hbase shell -n hbase/01_create_table.hbase
python3 src/load_hbase.py --method thrift
hbase shell -n hbase/02_verify.hbase 2>/dev/null | tee results/raw/02_verify.txt
hbase shell -n hbase/03_full_scan.hbase 2>/dev/null > results/raw/scan_emp.txt
hbase shell -n hbase/04_activity_b_queries.hbase 2>/dev/null | tee
results/raw/04_activity_b_queries.txt
python3 src/analyze.py --source thrift
python3 src/build_report.py
```